

Introduction to Neural Networks

Robin Jia & Xuezhe Ma
USC CSCI 544, Spring 2026
January 22, 2026

Recall: Linear Models & Simple Features

Training Data

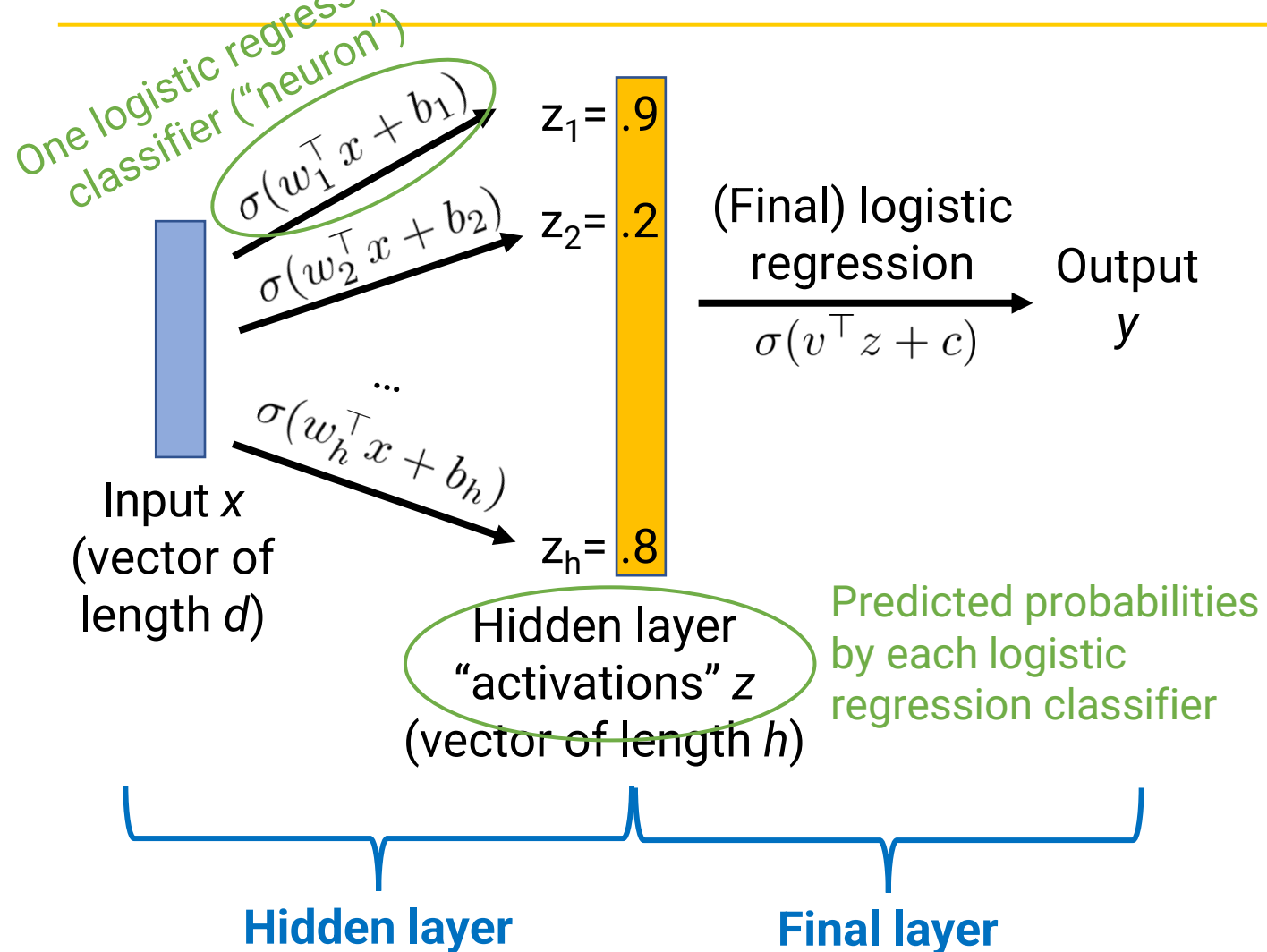
i	$y^{(i)}$	$x^{(i)}$
1	+1	it was good
2	-1	it was not good
3	-1	it was bad
4	+1	it was not bad

- Consider this simple training dataset
 - The unigram “*great*” appears in both positive and negative examples
 - So does “*bad*”
 - So does “*not*”
 - **With unigram features alone, can’t learn anything from this dataset!**
- To fit the dataset, need to expand the set of features (e.g., add bigram features)
 - **What if we could learn which features we need to add?**

Today's Plan

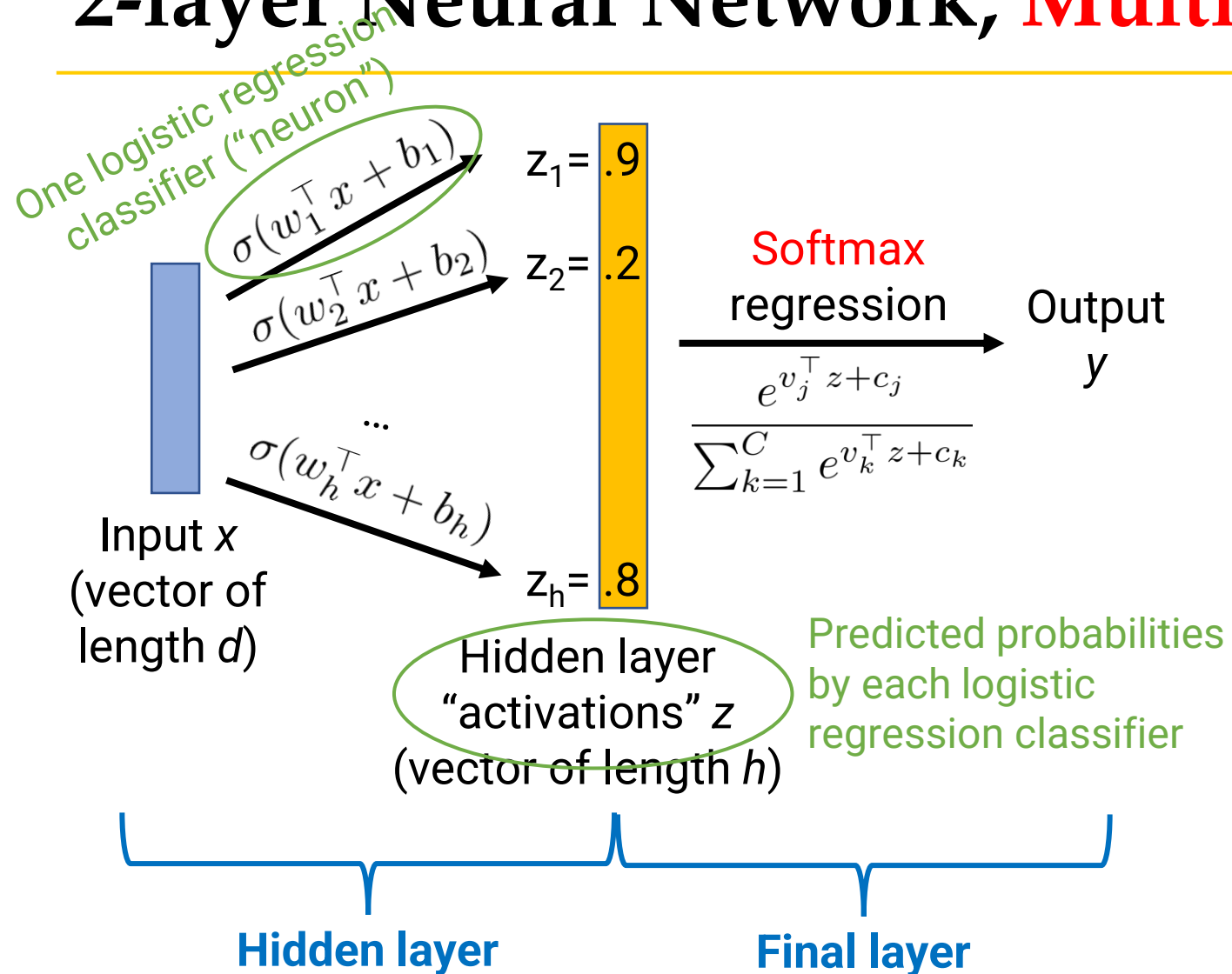
- Our First Neural Network
- Deep Learning as Learnable Non-Linear Functions
- Deep Learning as Feature Learning
- Deep Learning as Assembling Building Blocks
- Optimization for Deep Learning

2-layer Neural Network, Binary Classification



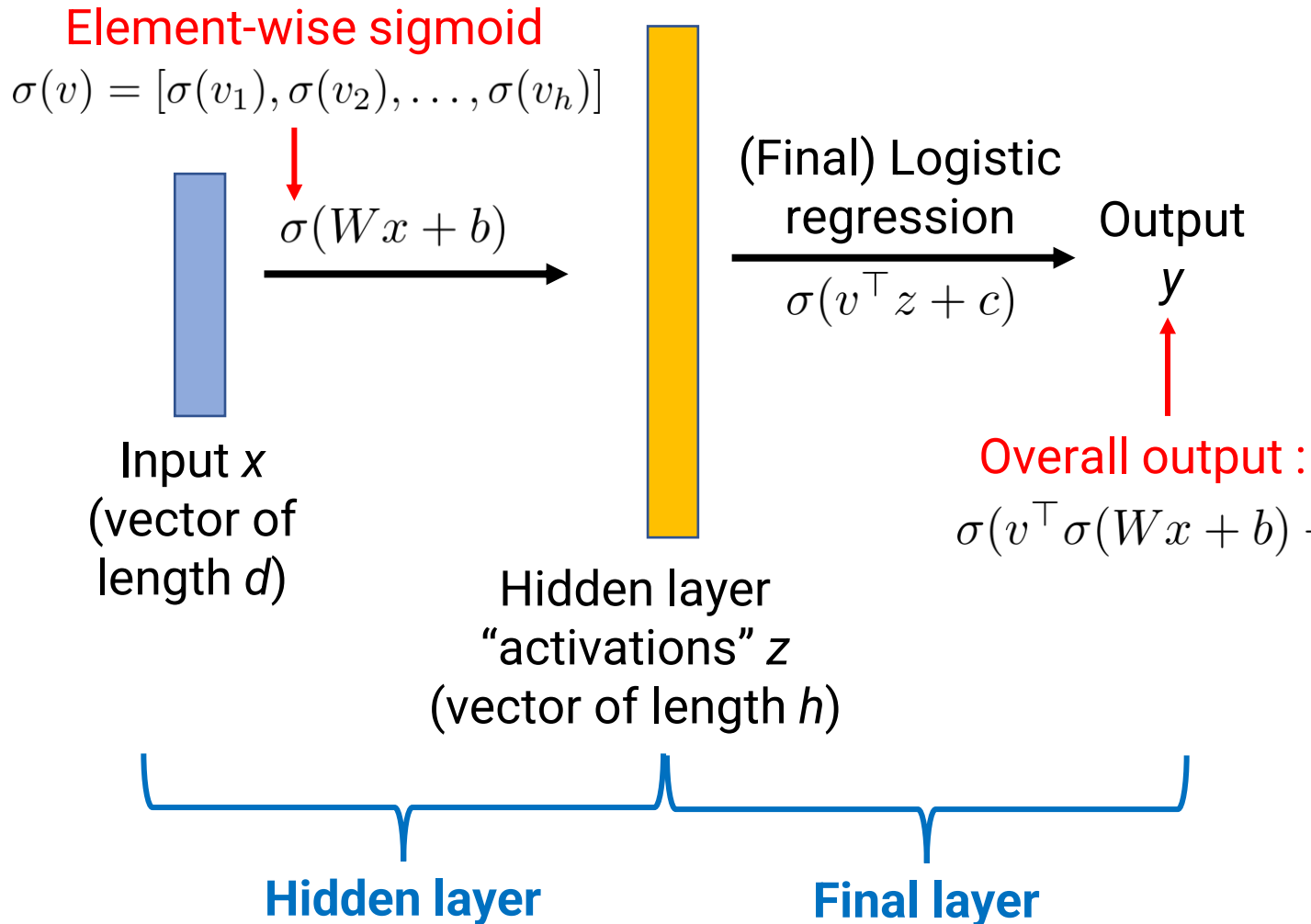
- Hidden layer = A bunch of logistic regression classifiers
 - Parameters: w_j and b_j for each classifier, for each $j=1, \dots, h$
 - h = number of neurons in hidden layer ("hidden nodes")
 - Produces "activations" = learned feature vector
- Final layer = linear model
 - For binary classification: logistic regression-like layer with weight vector v and bias c

2-layer Neural Network, Multi-Class Classification



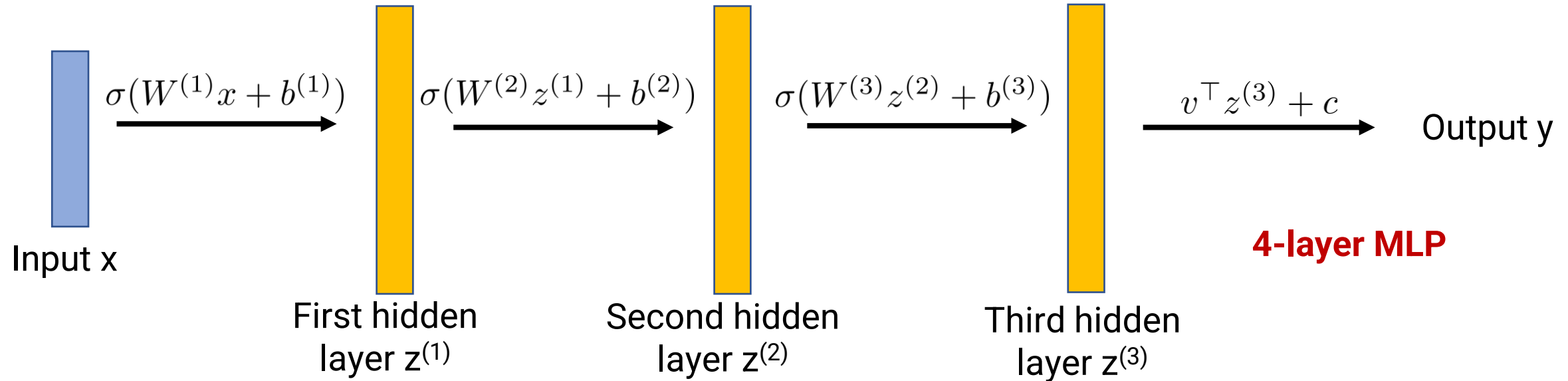
- Hidden layer = A bunch of logistic regression classifiers
 - Parameters: w_j and b_j for each classifier, for each $j=1, \dots, h$
 - h = number of neurons in hidden layer ("hidden nodes")
 - Produces "activations" = learned feature vector
- Final layer = linear model
 - For **multi-class classification**: linear model with weight vector v_k and bias c_k for each class k
 - **Only final layer changes when changing to a different task**

2-layer Neural Network in **Matrix Form**



- Hidden layer = A bunch of logistic regression classifiers
 - Parameters: w_j and b_j for each classifier, for each $j=1, \dots, h$
 - **Equivalently: matrix W ($h \times d$) and vector b (length h)**
 - h = number of neurons in hidden layer (“hidden nodes”)
 - Produces “activations”= learned feature vector
- Final layer = linear model
 - For binary classification: logistic regression-like layer with weight vector v and bias c
- **Parameters of model are $\theta = (W, b, v, c)$**

Multi-Layer Perceptron (MLP)



- What we saw so far is called a “2-layer perceptron”
- But we can add more layers!
 - Corresponds to more complex feature extractor
 - In practice, making networks “deeper” (more layers) often helps more than making them “wider” (more hidden units in each layer)
 - Layers are “fully connected” as each neuron depends on every neuron in previous layer



Powerful family
of non-linear
functions



Method for
learning features
from data



Set of building
blocks to create
complex models



Powerful family of non-linear functions

Neural Networks are Non-linear Functions

- Neural networks compute a **non-linear** function of input to make predictions
 - As a result, neural networks can learn many functions that linear models cannot
- In most other ways, neural networks are **very similar** to their linear counterparts!
 - E.g., Training is mostly the same

Training Neural Networks

Logistic Regression

- Model's score is

$$g(x) = w^\top x + b$$

- (Unregularized) loss function is

$$\frac{1}{n} \sum_{i=1}^n -\log \sigma(y^{(i)} \cdot g(x))$$

Divide by number
of examples to
view this as an
average across
dataset

Training objective for both types of models:

$$\frac{1}{n} \sum_{i=1}^n \ell(y^{(i)}, g(x^{(i)})), \text{ where } \ell(y, u) = -\log \sigma(y \cdot u)$$

To make this softmax
regression, just change ℓ

Binary Classification w/ Neural Networks

- Model's score is

$$g(x) = v^\top \sigma(Wx + b) + c$$

- **Use same loss function**, in terms of g !

$$\frac{1}{n} \sum_{i=1}^n -\log \sigma(y^{(i)} \cdot g(x))$$

Training Neural Networks

General loss function: $\frac{1}{n} \sum_{i=1}^n \ell \left(y^{(i)}, g(x^{(i)}) \right)$

Model's output, depends on all model parameters θ (includes all layers)

- How to minimize? Gradient Descent!

$$\theta \leftarrow \theta - \eta \cdot \underbrace{\frac{1}{n} \sum_{i=1}^n \nabla_{\theta} \ell \left(y^{(i)}, g(x^{(i)}) \right)}_{\text{Average of per-example gradients}}$$

Average of per-example gradients

- In practice, use a variant of traditional gradient descent
 - (Will discuss later)

Importance of “Non-linearities”

With sigmoid,
overall output is:

$$v^T \sigma(Wx + b) + c$$

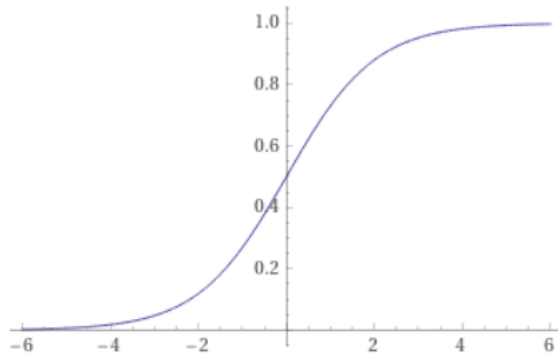
Without sigmoid,
overall output is:

$$\begin{aligned} &v^T (Wx + b) + c \\ &= (v^T W)x + (v^T b + c) \end{aligned}$$

- Having the sigmoid is very important!
- What if we skipped the sigmoid?
- Result: Just another way to write a linear function!
 - New “weight” is $v^T W$
 - New “bias” is $v^T b + c$
- Having a **simple non-linear function** (like sigmoid) between the two linear operations enables us to learn a **complex non-linear function**!

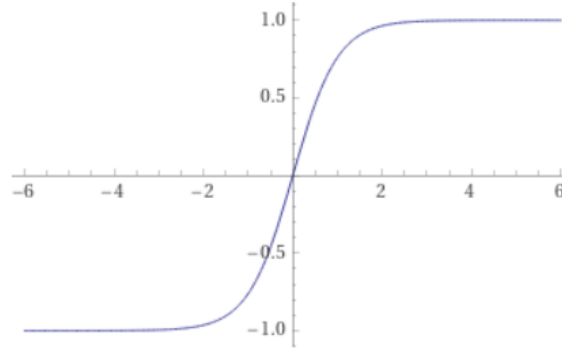
Options for Non-linearities

Sigmoid



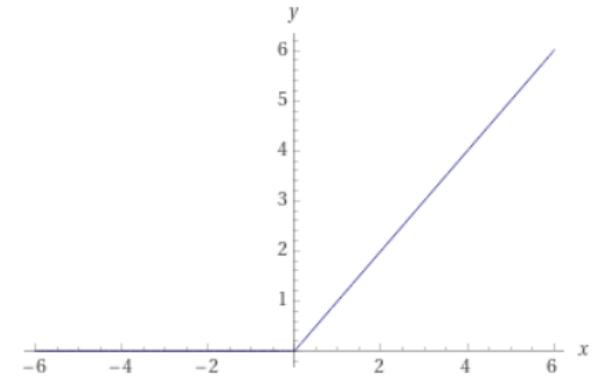
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Tanh



$$\tanh(z) = \frac{e^{2z} - 1}{e^{2z} + 1}$$

ReLU



$$\text{ReLU}(z) = \max(z, 0)$$

- Many options work, just must be differentiable (for gradient descent)
 - Sometimes called “activation functions” or just “non-linearities”
- In practice: tanh and ReLU often preferred
 - Tanh: Better than sigmoid because outputs centered around zero
 - ReLU: Very fast to compute

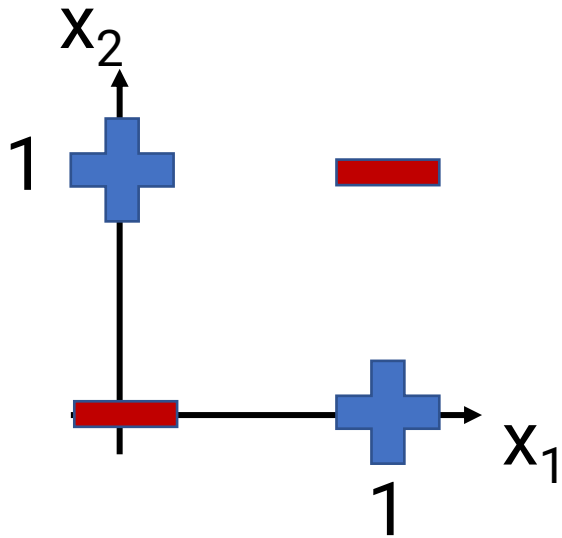
Summary: Neural Networks as Non-linear Functions

- Neural Networks are a type of **learnable non-linear function**
 - Contrast with previous models, which are learnable linear functions
- Constructed from multiple layers, each of which have their own learnable parameters
- Learning process works the same way as for linear models

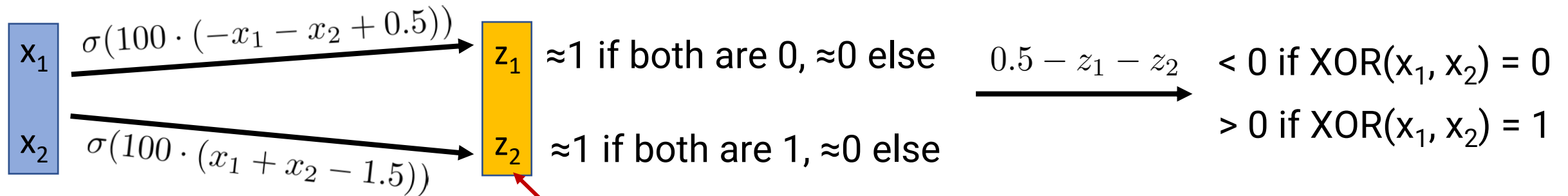


Method for learning features from data

Non-linearities make NN's more expressive



- XOR: Classic binary classification problem that can't be solved by linear classifier
- A 2-layer neural network can solve it!
 - I can choose values for the parameters that lead to perfect classification on this dataset
 - Conclusion: Neural networks are **more expressive** than linear models



**From perspective of final (linear) layer,
these are new features we've learned!**

Modeling Negation with an MLP

- Suppose we have 3 input features:

- x_1 = is there a positive word?
- x_2 = is there a negative word?
- x_3 = is there a “not”?

*what a bad
movie*

0
1
0

*I did not think it
was good*

1
0
1

- An MLP can learn these features:

- $z_1 \approx 1$ if $x_1 + x_3 > 1.5$, else ≈ 0 *Detects “not” + positive word*
- $z_2 \approx 1$ if $x_2 + x_3 > 1.5$, else ≈ 0 *Detects “not” + negative word*
- $z_3 \approx 1$ if $x_1 - x_3 > 0.5$, else ≈ 0 *Detects positive word without “not”*
- $z_4 \approx 1$ if $x_2 - x_3 > 0.5$, else ≈ 0 *Detects negative word without “not”*

- Final layer: Return $-10z_1 + 10z_2 + 10z_3 - 10z_4$
- Takeaway: We don’t have to manually add more features (bigrams)
- MLP can learn more complex features from simple ones!

Summary: Neural Networks as Feature Learners

- Neural networks **learn new features from data**
- Each learned feature is the output of a linear classifier using the features provided as input
- These classifiers can be trained to produce features that help the final layer make good predictions



Set of building blocks to create complex models

Deep Learning as a Set of Building Blocks

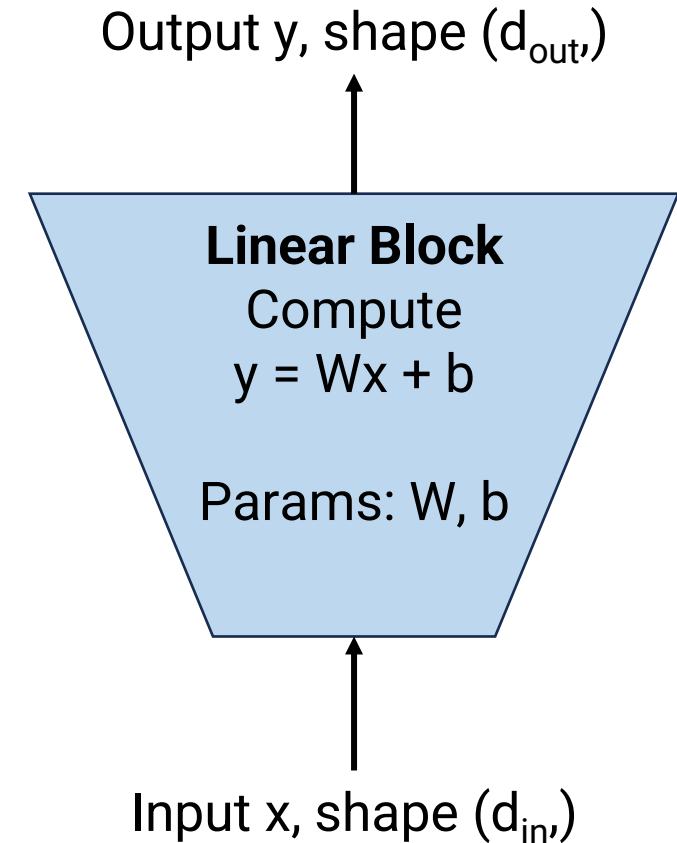
- Neural Network = Many “layers” stacked on top of each other
 - Each “layer” takes in some input and computes some output
 - Simplest layers are basic building blocks, can build more complex layers from those
 - Arrangement of layers is called an “architecture”
- Deep Learning: **Design suitable neural architectures with various reusable building blocks**
 - This is the view of most deep learning programming libraries like pytorch



The Basic “Building Blocks”

(1) Linear Block

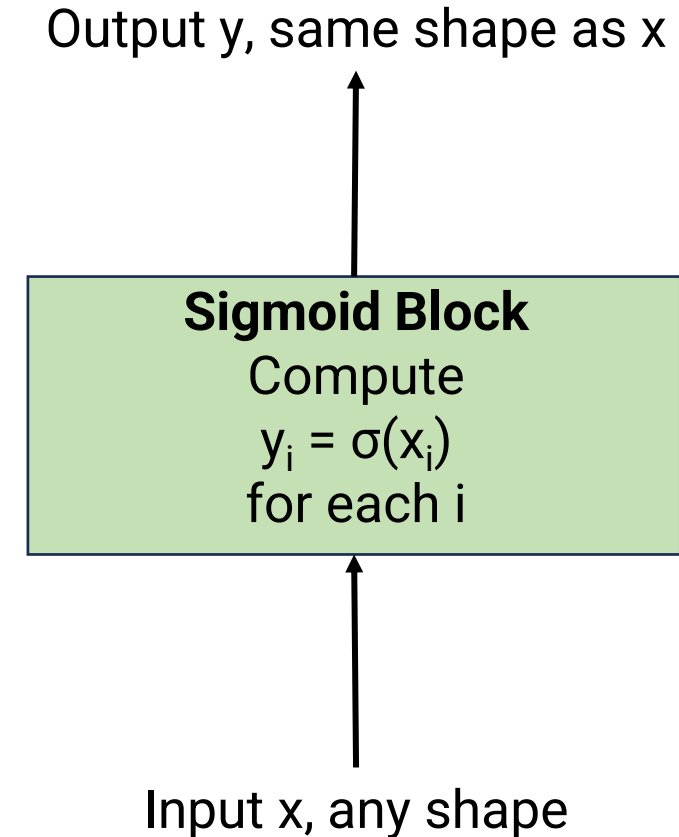
- Input x : Vector of dimension d_{in}
- Output y : Vector of dimension d_{out}
- Formula: $y = Wx + b$
- Parameters
 - W : $d_{out} \times d_{in}$ matrix
 - b : d_{out} vector
- In pytorch: `nn.Linear()`



The Basic “Building Blocks”

(2) Non-linearity Block

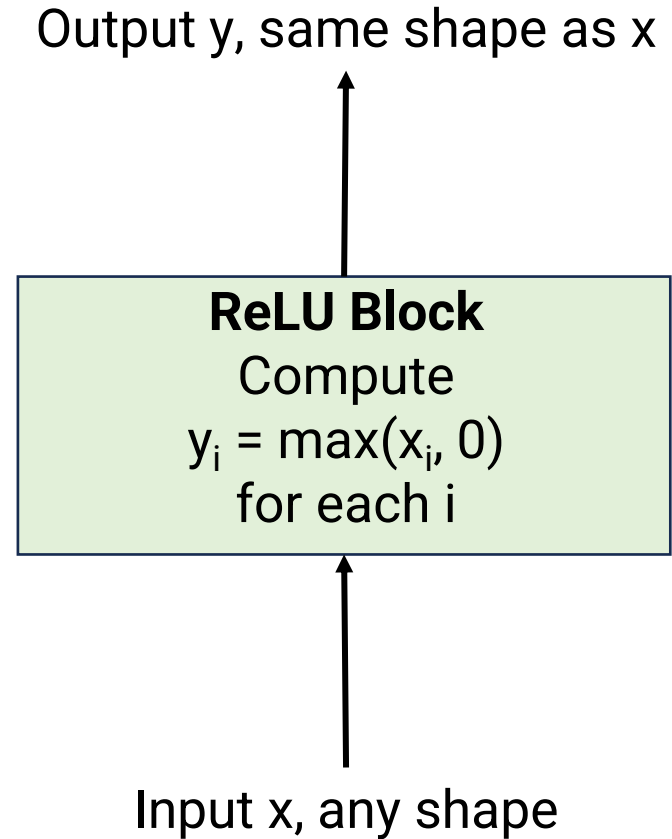
- Input x : Any number/vector/matrix
- Output y : Number/vector/matrix of same shape
- Possible formulas:
 - Sigmoid: $y = \sigma(x)$, elementwise
 - Tanh: $y = \tanh(x)$, elementwise
 - Relu: $y = \max(x, 0)$, elementwise
- Parameters: None
- In pytorch: `torch.sigmoid()`, `nn.functional.relu()`, etc.



The Basic “Building Blocks”

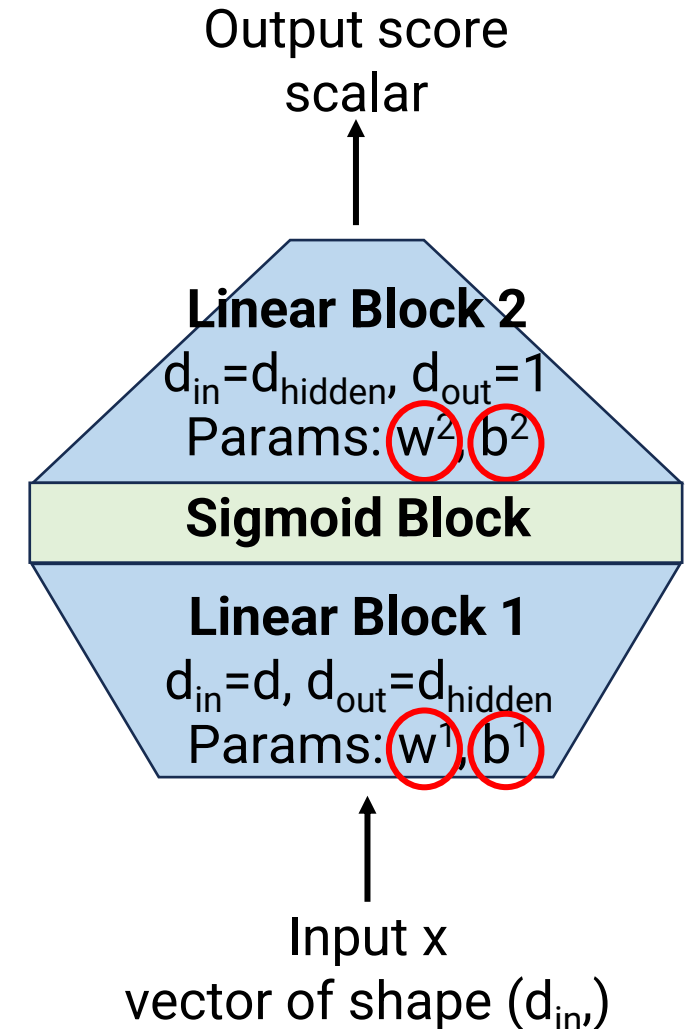
(2) Non-linearity Block

- Input x : Any number/vector/matrix
- Output y : Number/vector/matrix of same shape
- Possible formulas:
 - Sigmoid: $y = \sigma(x)$, elementwise
 - Tanh: $y = \tanh(x)$, elementwise
 - Relu: $y = \max(x, 0)$, elementwise
- Parameters: None
- In pytorch: `torch.sigmoid()`, `nn.functional.relu()`, etc.



Building a 2-layer MLP

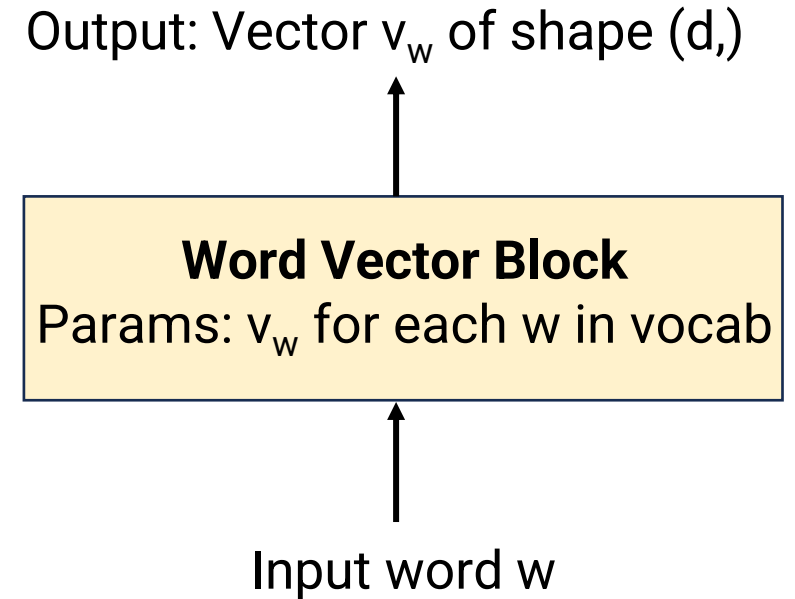
- A 2-layer MLP is a stack of 3 building blocks
 - Linear Layer #1
 - Non-linearity (e.g., sigmoid)
 - Linear Layer #2
- Composing these 3 blocks yields a map from d_{in} -dimensional vectors to scalars
- Parameters of model = Total parameters across all blocks
- Training = Update all parameters at same time via gradient descent



Word Vector Building Block

(3) Word Vector Block

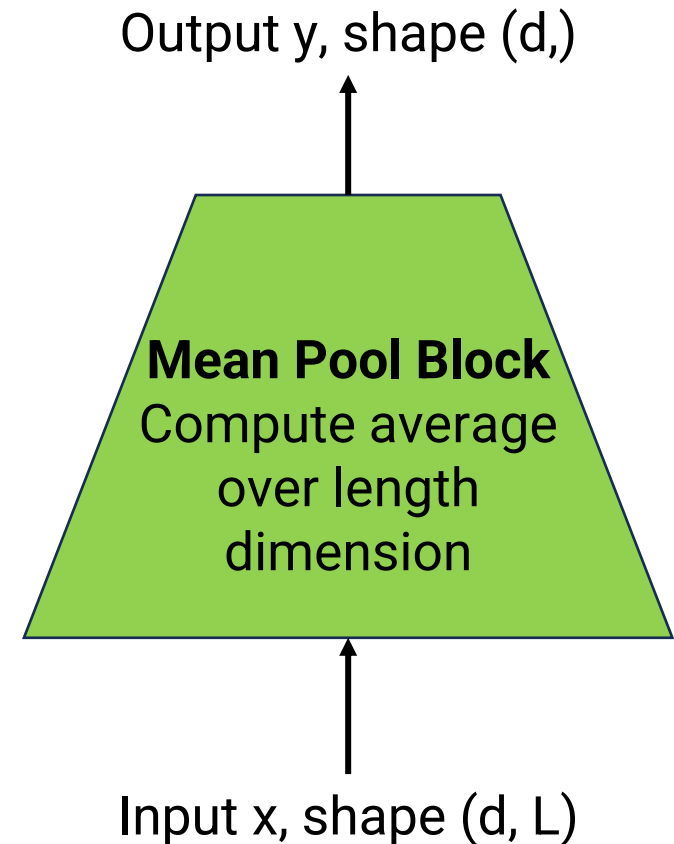
- Input w : A word (from our vocabulary)
 - Can also take a list of words
- Output: A vector of length d
 - If input is list of L words, outputs matrix of shape (d, L)
- Formula: Return `word_vecs[w]`
- Parameters:
 - For each word w in vocabulary, there is a word vector parameter v_w of shape d
 - $|V| * d$ total parameters needed



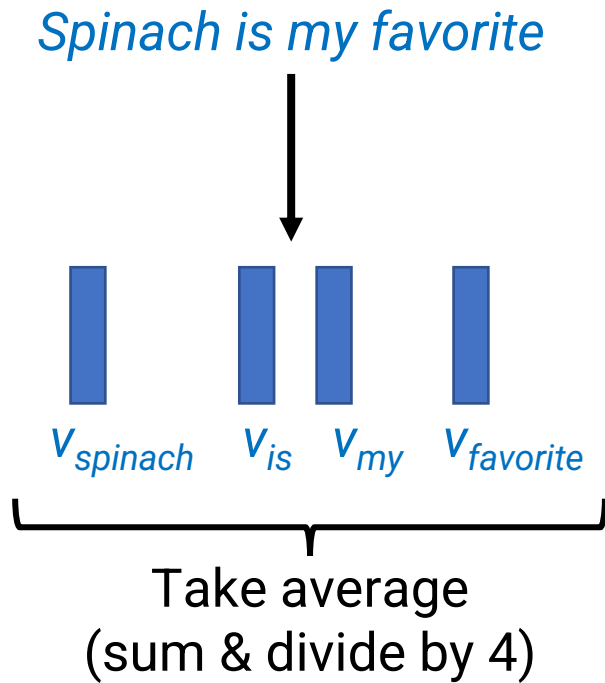
Mean Pooling Building Block

(5) Mean Pooling Block

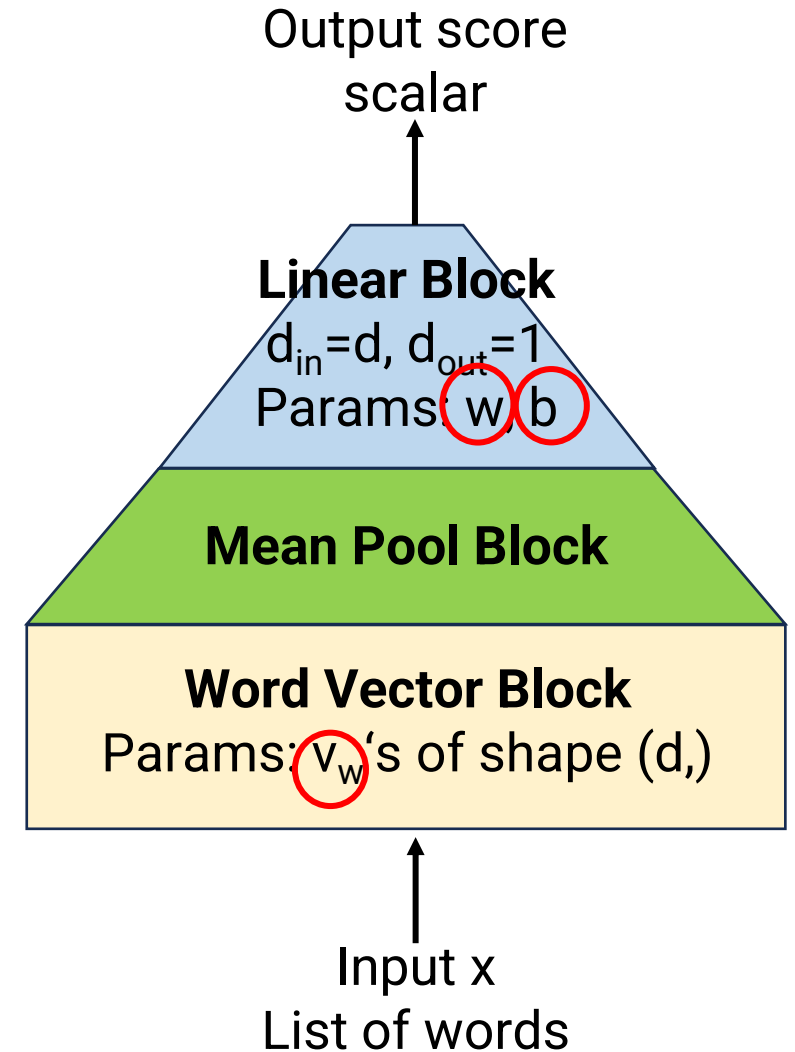
- Input x: Matrix of shape (d, L)
 - d = dimension, L = length
- Output y: Tensor of dimension d
- Formula: Average all the vectors along the “L” dimension
- Parameters: None
- In pytorch: `torch.mean()`



Re-making the Word Vector Classifier

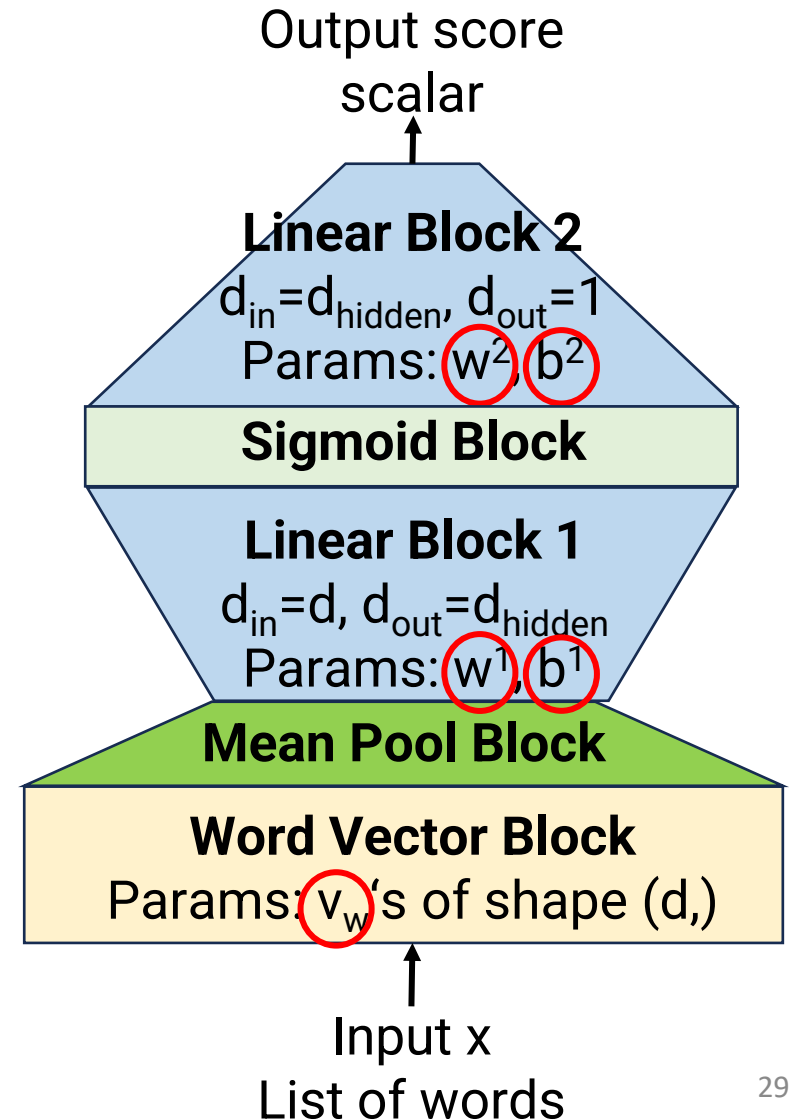


- Total classifier is:
 - Map words to word vectors
 - Average them
 - Apply a linear transformation



A Neural Network Text Classifier

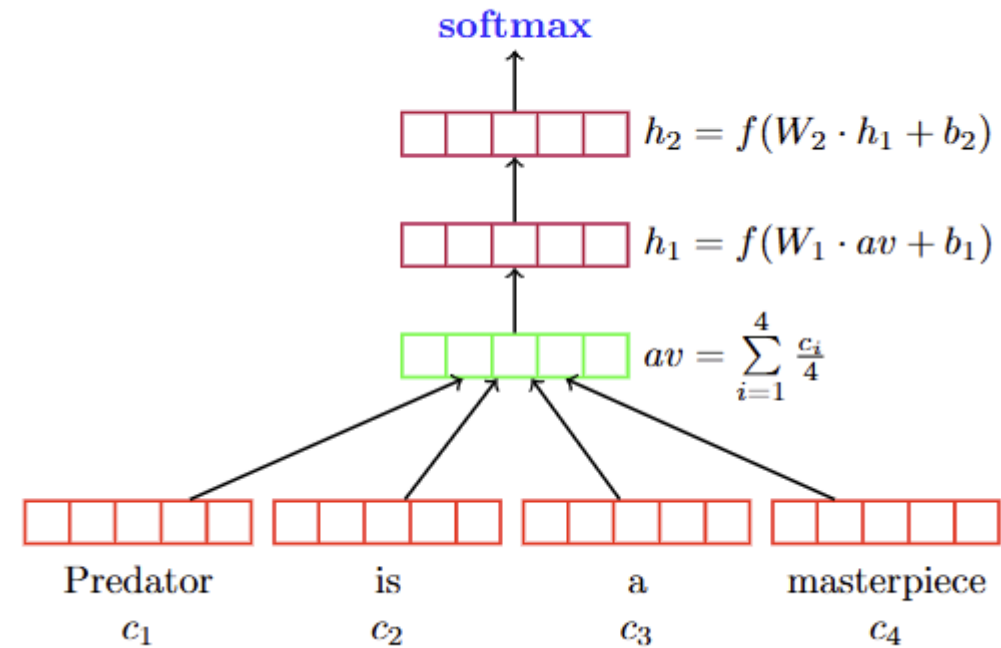
- We can easily make this a neural network by replacing the single linear block with an MLP!
- **Can**: Create features that combine multiple words
 - E.g., “bat” and “wing” both appeared
- **Cannot**: Take word order into account



Deep Averaging Network

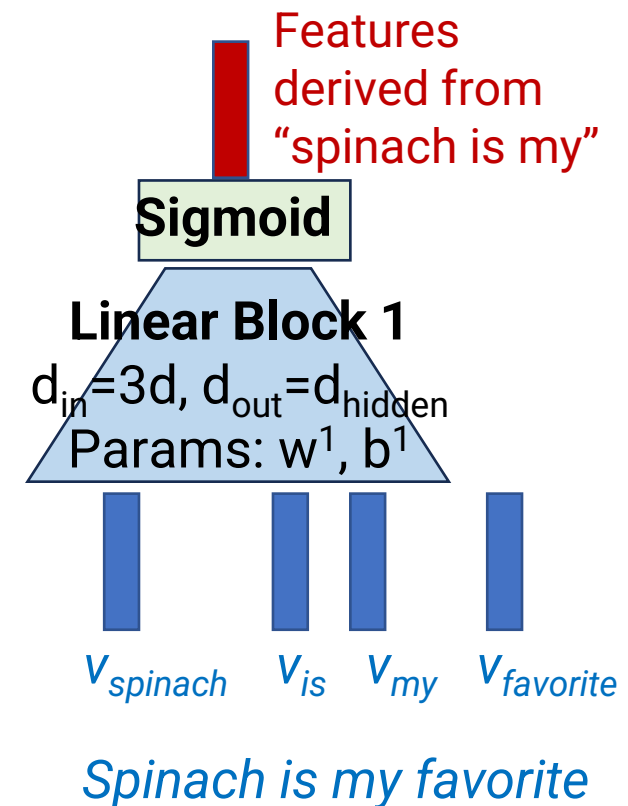
- This model actually has a name: Deep Averaging Network
- Paper shows this outperforms methods like Naïve Bayes on sentiment analysis

DAN



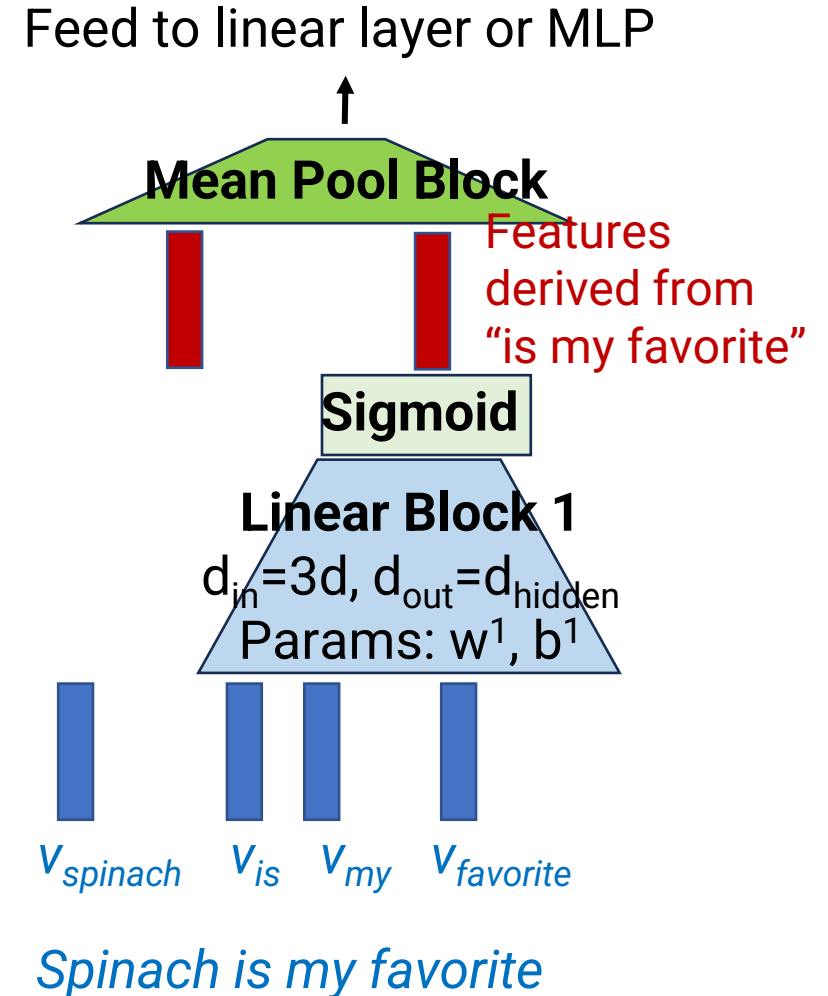
Beyond Bag of Words

- Recall: We did mean pooling because documents have variable length
- But there is something we can do first!
 - Train a neural network to look at **fixed**-size windows (e.g., 3 words)
 - Run that same network on **every** such window
 - Mean pool the resulting feature outputs



Beyond Bag of Words

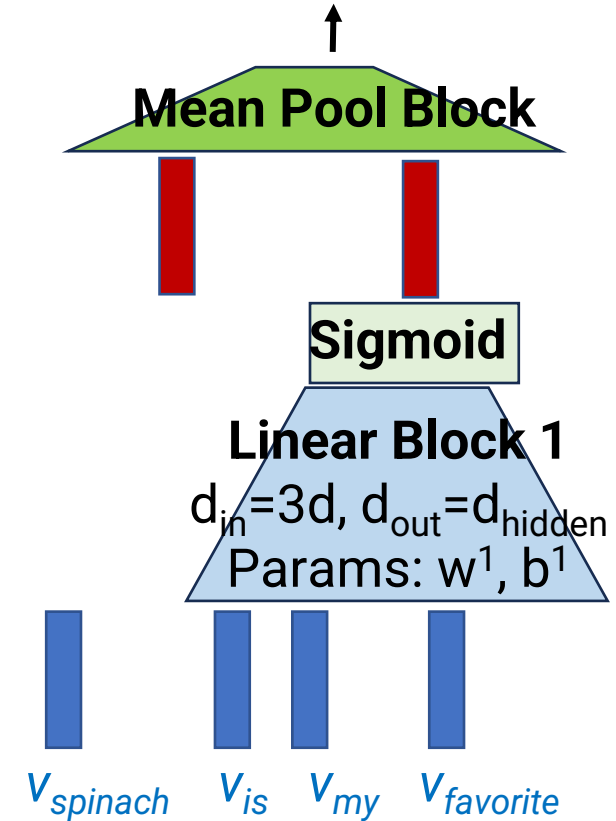
- Recall: We did mean pooling because documents have variable length
- But there is something we can do first!
 - Train a neural network to look at **fixed**-size windows (e.g., 3 words)
 - Run that same network on **every** such window
 - Mean pool the resulting feature outputs
- We just created a **convolutional neural network**!



Convolutional Neural Networks

- CNN's can create neural “bag of n-grams” features
 - Each intermediate feature vector can combine information from all words in the window
- Comparison with n-gram indicator features
 - Word vectors help us process unseen n-grams!
 - Very useful because chances of seeing an exact sequence of {3, 5, ...} words is very low

Feed to linear layer or MLP



Summary: Deep Learning as Building Blocks

- Power of deep learning: You can **stack building blocks together any way you want**
- NLP: First map words to word vectors, then put other blocks on top
- Convolutional (sliding window) networks can learn to construct features from fixed-length phrases



Today's Plan

- Our First Neural Network
- Deep Learning as Learnable Non-Linear Functions
- Deep Learning as Feature Learning
- Deep Learning as Assembling Building Blocks
- **Optimization for Deep Learning**

Stochastic gradient descent

General loss function: $\frac{1}{n} \sum_{i=1}^n \ell(y^{(i)}, g(x^{(i)}))$

Model's output, depends on parameters θ

Gradient Descent

$$\theta \leftarrow \theta - \eta \cdot \underbrace{\frac{1}{n} \sum_{i=1}^n \nabla_{\theta} \ell(y^{(i)}, g(x^{(i)}))}_{\text{Average of per-example gradients}}$$

- Disadvantage: 1 update is $O(n)$ time
 - What if dataset is very large?
- Idea: Approximate with sample mean

Stochastic Gradient Descent

1. Sample a *batch* B of examples from the training dataset
2. Do the update

$$\theta \leftarrow \theta - \eta \cdot \underbrace{\frac{1}{|B|} \sum_{(x,y) \in B} \nabla_{\theta} \ell(y, g(x))}_{\text{Sample mean within batch}}$$

Sample mean within batch

Stochastic gradient descent

In practice, partition training set into batches:

for $t = 1, \dots, T$: Each t (i.e., each pass over the dataset) is called one “epoch”

Randomly partition training examples into batches $B_1, \dots, B_k$

for $i = 1, \dots, k$:

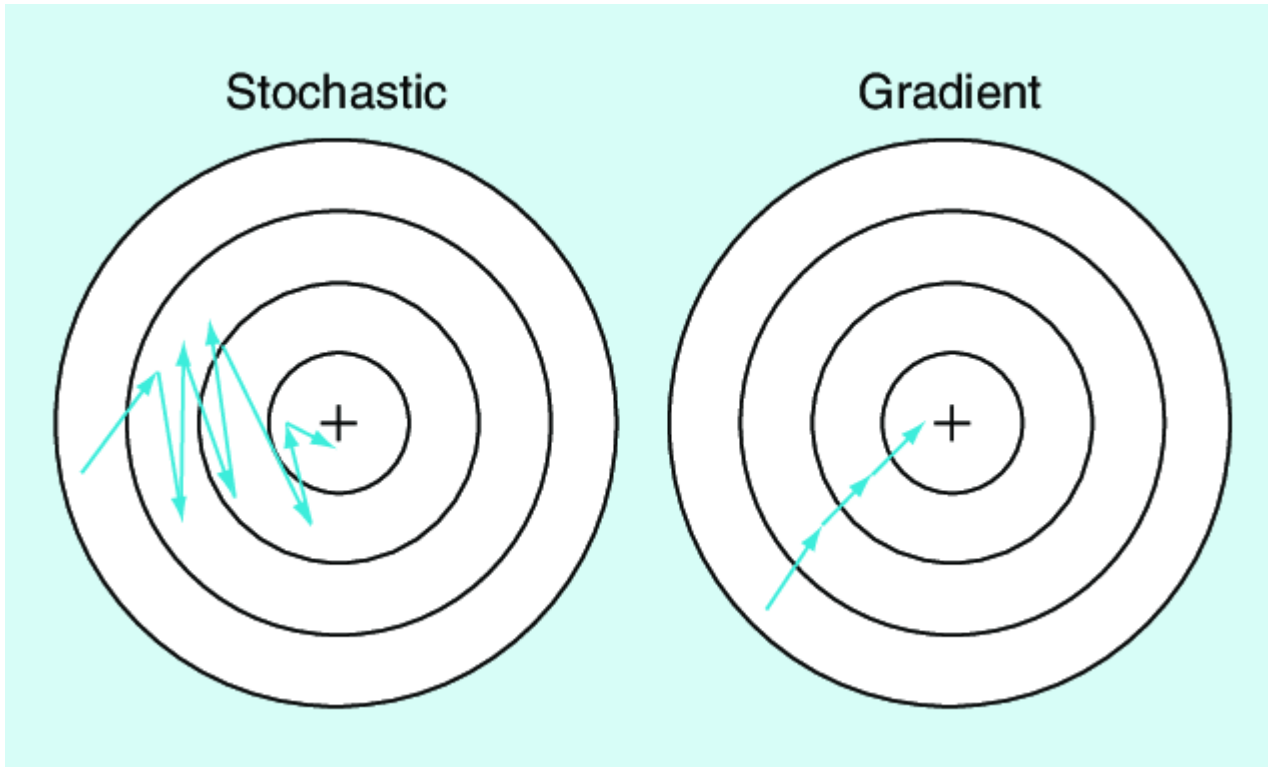
$$\theta \leftarrow \theta - \eta \cdot \frac{1}{|B_i|} \sum_{(x,y) \in B_i} \nabla_{\theta} \ell(y, g(x))$$

Update based on sample mean within current batch

How many batches? Desired “batch size” (# examples/batch) is another hyperparameter to tune

- Larger batch size = more accurate gradient, but slower
- Smaller batch size = faster, but may wander in suboptimal directions
- SGD is most useful when training data is large, computing full gradient is expensive

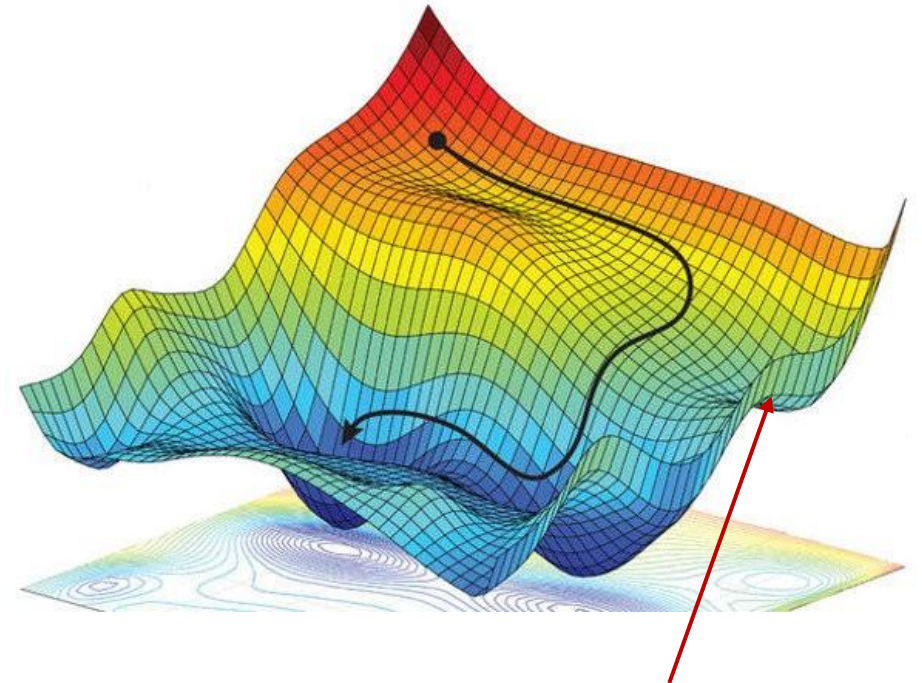
Stochastic gradient descent



- SGD: Each parameter update is only “approximately” going towards the minimum
- But given enough time, you’ll end up in (almost) the same place
 - Plus each step is much faster!

Non-convexity During Training

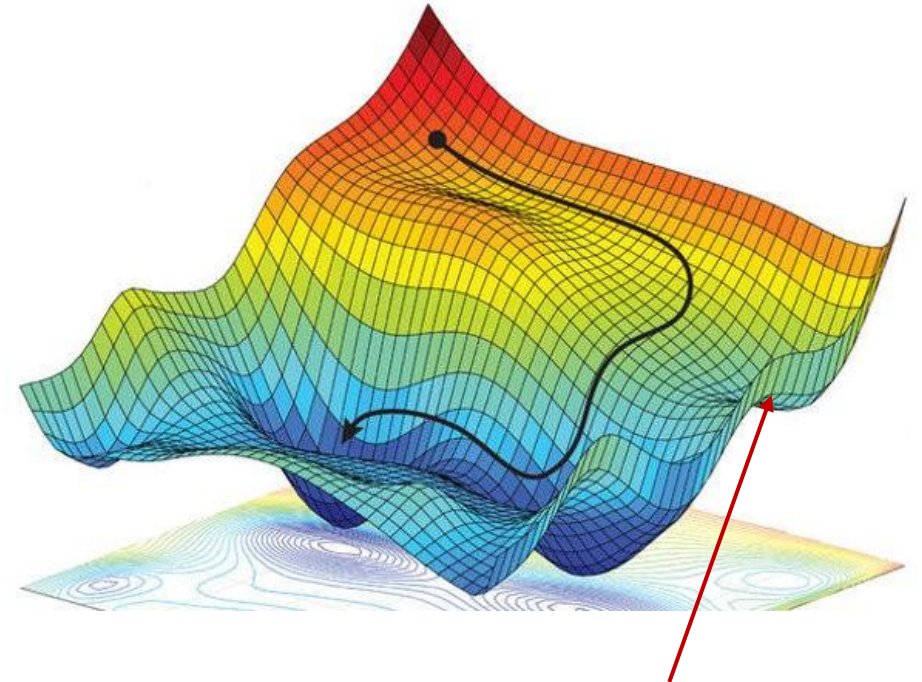
- Linear models were **convex**
 - All local optima are global optima
 - All reasonable optimization methods will find a global optimum
- Neural Networks are **non-convex**
 - Very hard to find global optimum
 - Different optimization techniques will converge to **different local optima**, some of which are much better than others
 - **Choice of optimization method really matters!**



Local minimum
Gradient descent can get stuck here!

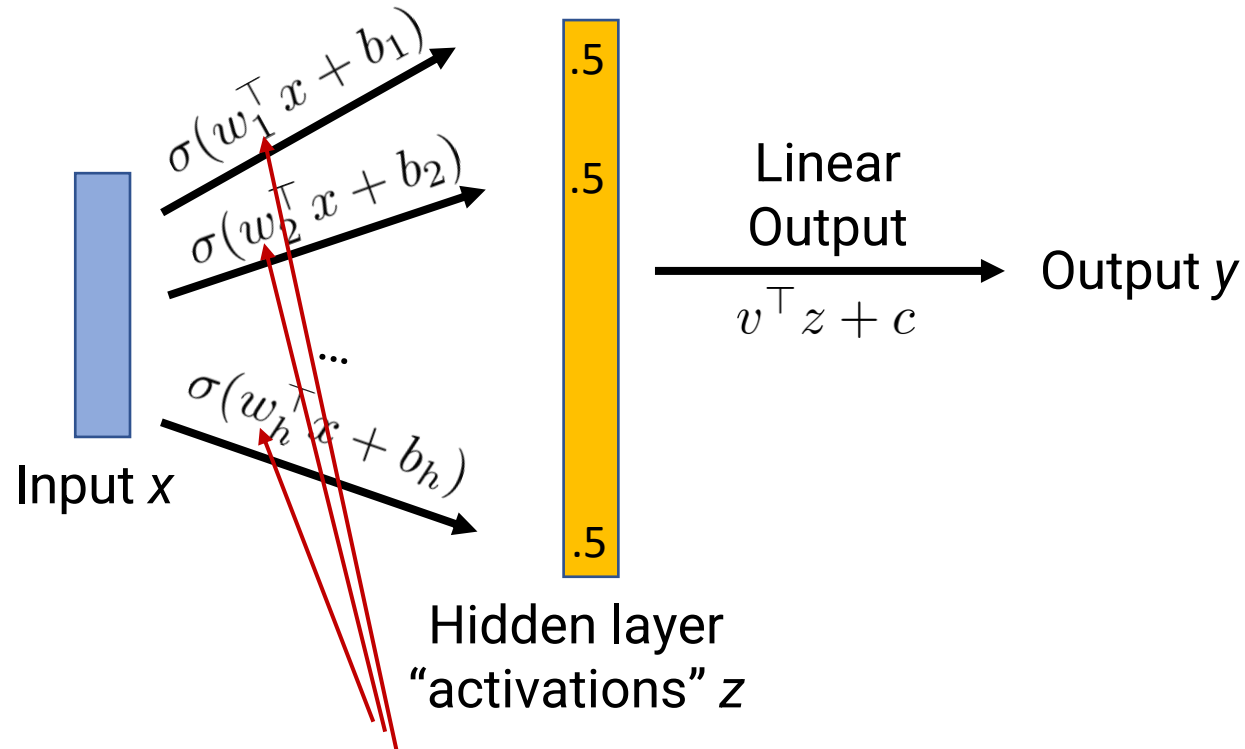
Initialization

- For convex problems (e.g. logistic regression), initialization doesn't matter much for final result
 - We just initialize parameters to all 0's
- For neural networks, initialization matters a lot!
 - Optimization problem is non-convex
 - Where you start determines what parameters you learn



Local minimum
If you initialize here,
you get stuck here!

The problem with all-0's initialization



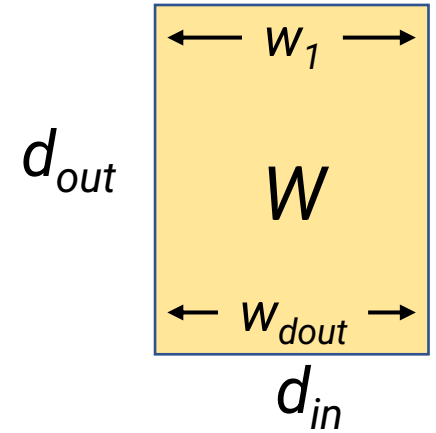
If every w_j starts as 0 vector,
gradient update to each w_j will be the same

- What if we initialize with all 0's?
- Problem: Symmetry
 - All hidden units start out the same, so gradients will be the same for each
 - Thus, all hidden units will stay the same!
- **Solution: Initialize with small random values**
 - This breaks the symmetry, as different hidden units start off computing slightly different features

How to initialize neural networks

- Several options

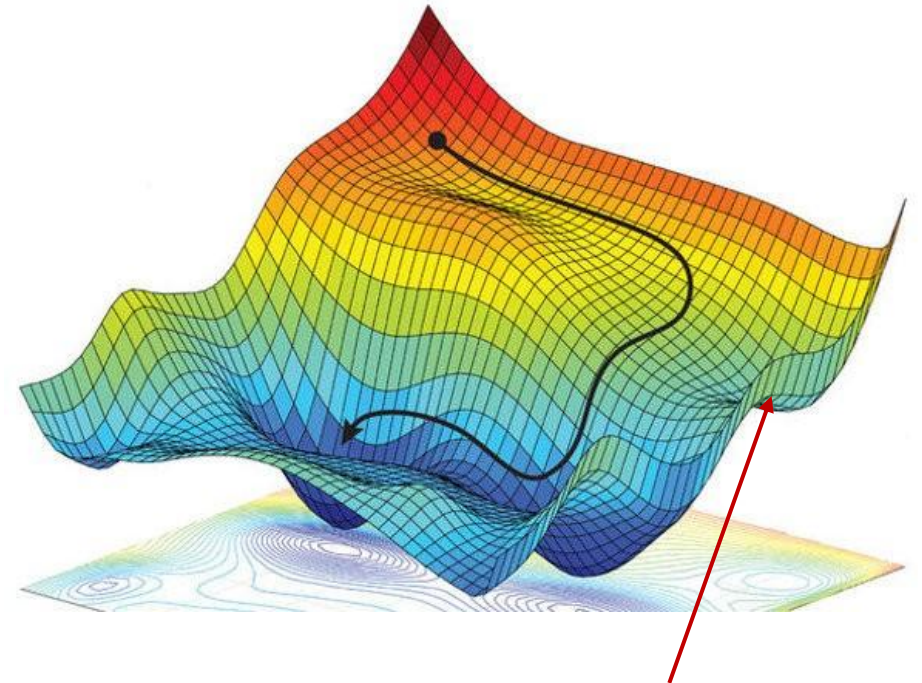
- He initialization: $\text{Normal}\left(0, \frac{2}{d_{in}}\right)$ (mean 0, variance $2/d_{in}$)
- Xavier initialization: $\text{Normal}\left(0, \frac{2}{d_{in} + d_{out}}\right)$ Also divides by d_{out}
(But usually d_{in} and d_{out} are similar size)
- Pytorch default: $\text{Uniform}\left(-\frac{1}{\sqrt{d_{in}}}, \frac{1}{\sqrt{d_{in}}}\right)$ Uniform avoids large outliers
Note: Variance is proportional to $1/d_{in}$



- Usually you don't tune these as hyperparameters, just use defaults

Importance of Learning Rate

- For convex problems (e.g. logistic regression), **learning rate** doesn't change final result very much
 - All reasonable values converge to the same final answer
- For neural networks, learning rate matters a lot!



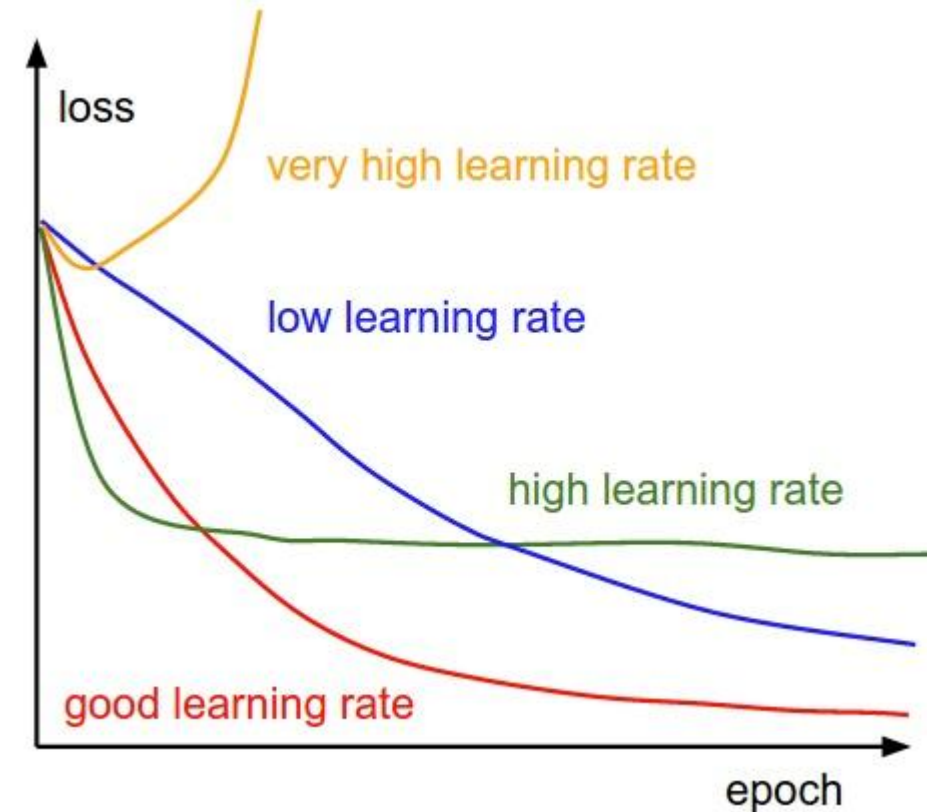
Local minimum

Get stuck here if learning rate
is too small

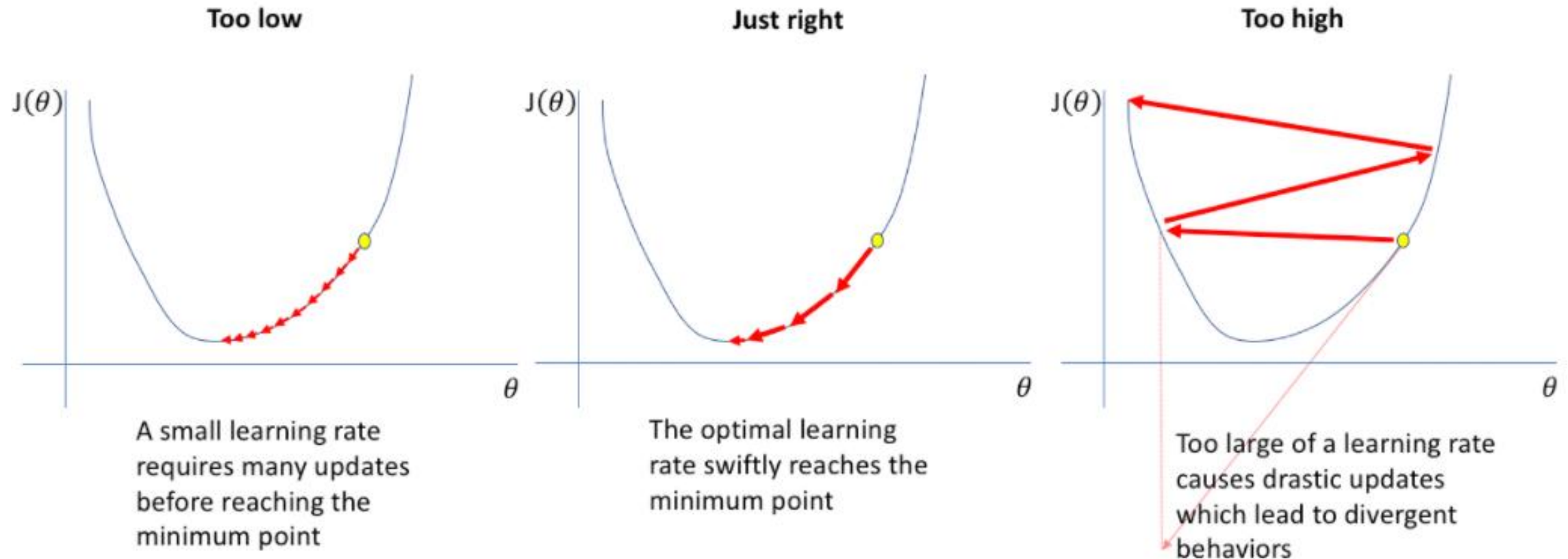
Importance of Learning Rate

$$\theta \leftarrow \theta - \boxed{\eta} \cdot \frac{1}{|B|} \sum_{(x,y) \in B} \nabla_{\theta} \ell(y, g(x))$$

- Too small: Can't take big enough steps, don't converge fast enough, can get stuck in "flat" regions
- Too large: Steps are too large, too erratic and doesn't converge
- Need to carefully tune learning rate

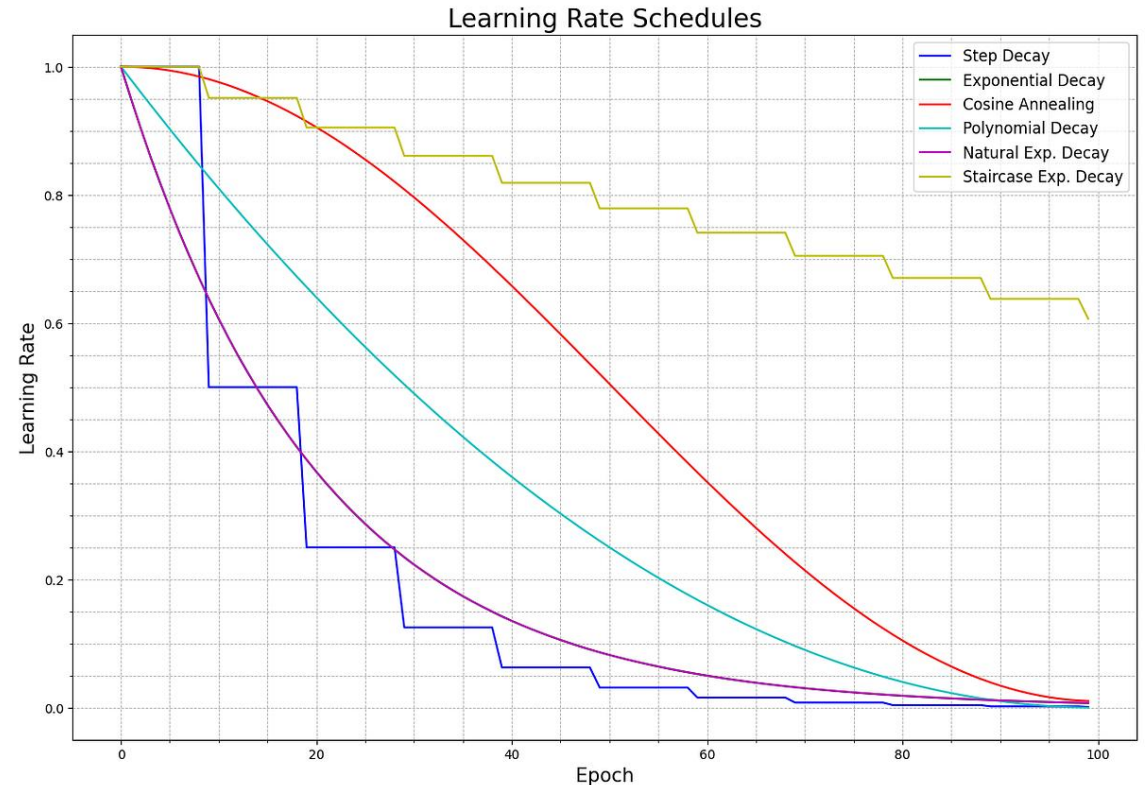


Importance of Learning Rate



Learning Rate Schedules

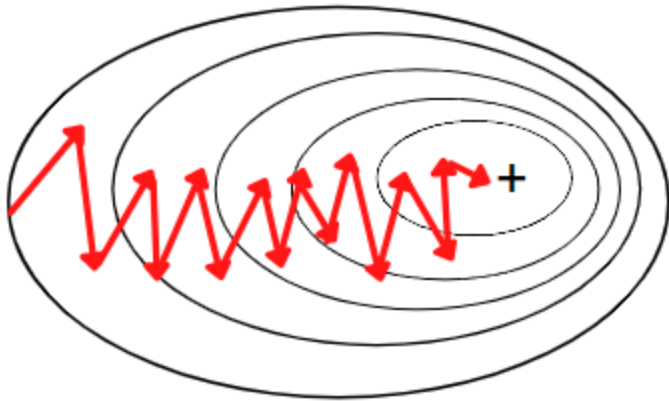
- Early on: We're far from the optimum, want to take large steps
- Later on: We're close to the optimum, take small steps so we don't "overshoot"
- Solution: Start with large learning rate, make it smaller over time ("decay")



Challenges for SGD

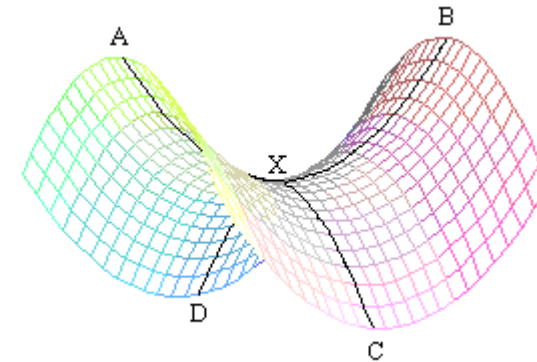
Problem #1: Ill-Conditioned loss

- Surface is very curved/steep in some directions, but shallow in others
- High learning rate: Zig-zag along steep direction
- Low learning rate: Move slowly along shallow direction



Problem #2: Saddle Points

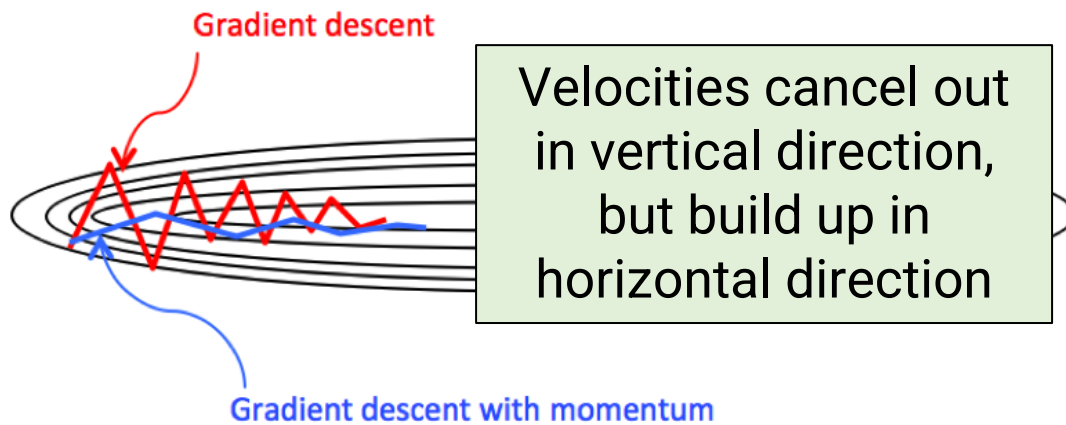
- Saddle Point = Area that is locally flat, but neither minimum nor maximum
- SGD can get stuck here because gradient is close to 0 $\rightarrow$ take small steps



Two Ways to Improve SGD

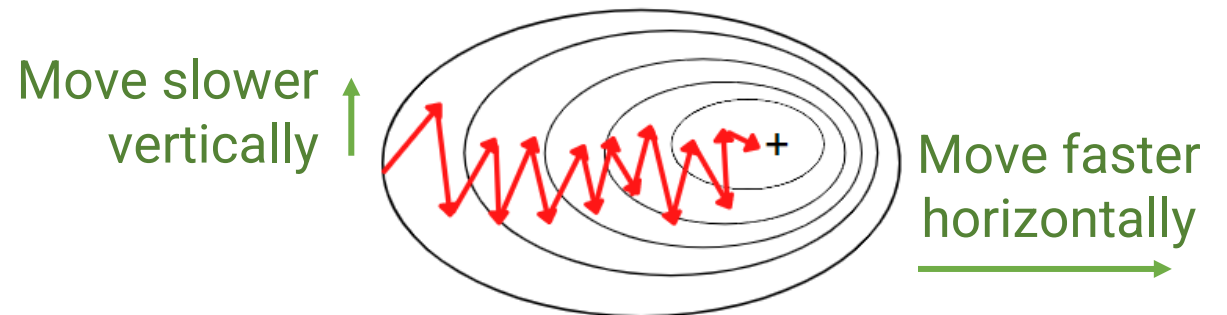
Idea #1: Momentum

- Maintain some “momentum”
- Each batch applies “force” in direction of gradient
- Then move in the direction of your current momentum



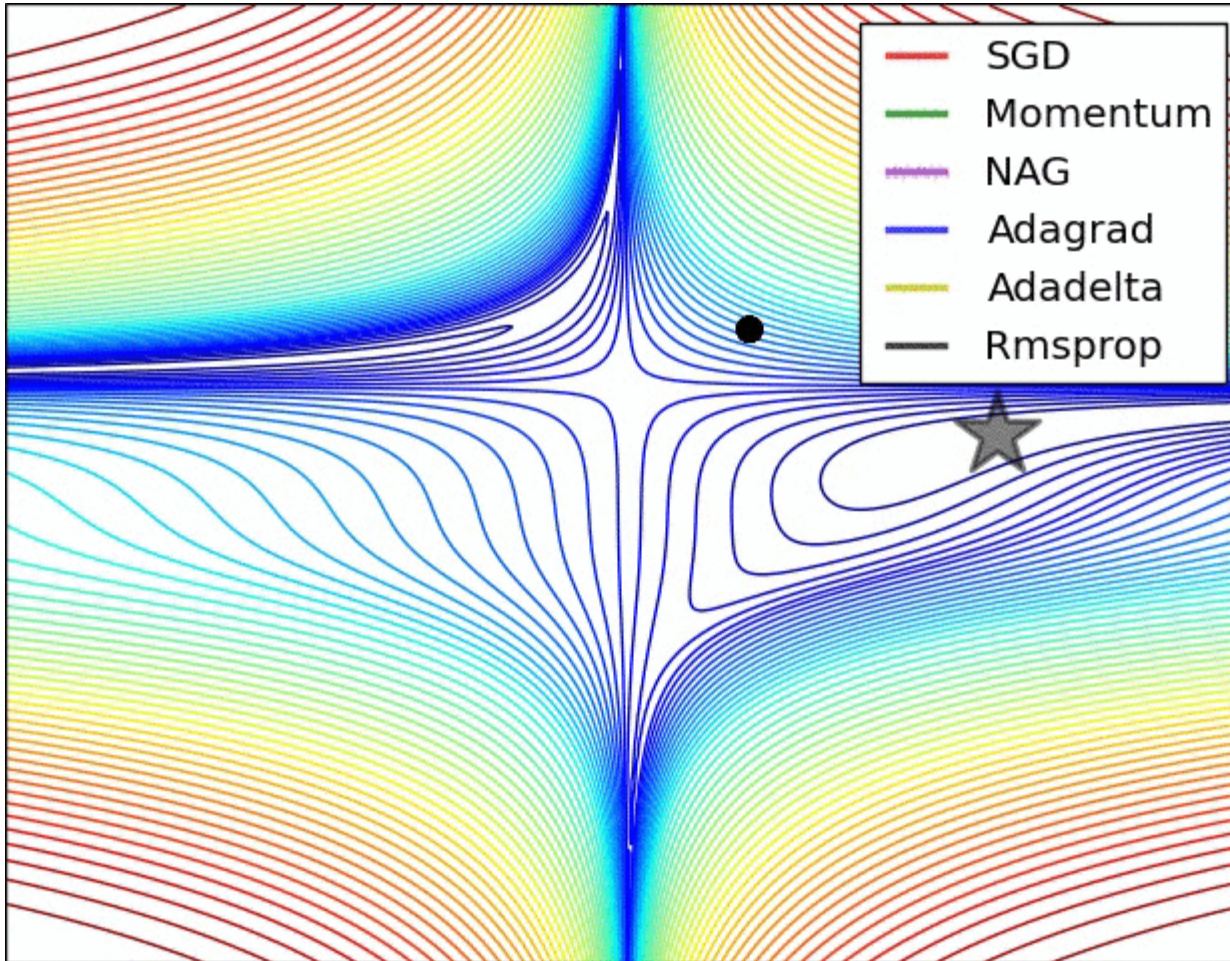
Idea #2: Per-Parameter Learning Rate

- Landscape may be...
 - “Steep” in some directions: Need smaller step
 - “Flat” in other directions: Need larger step



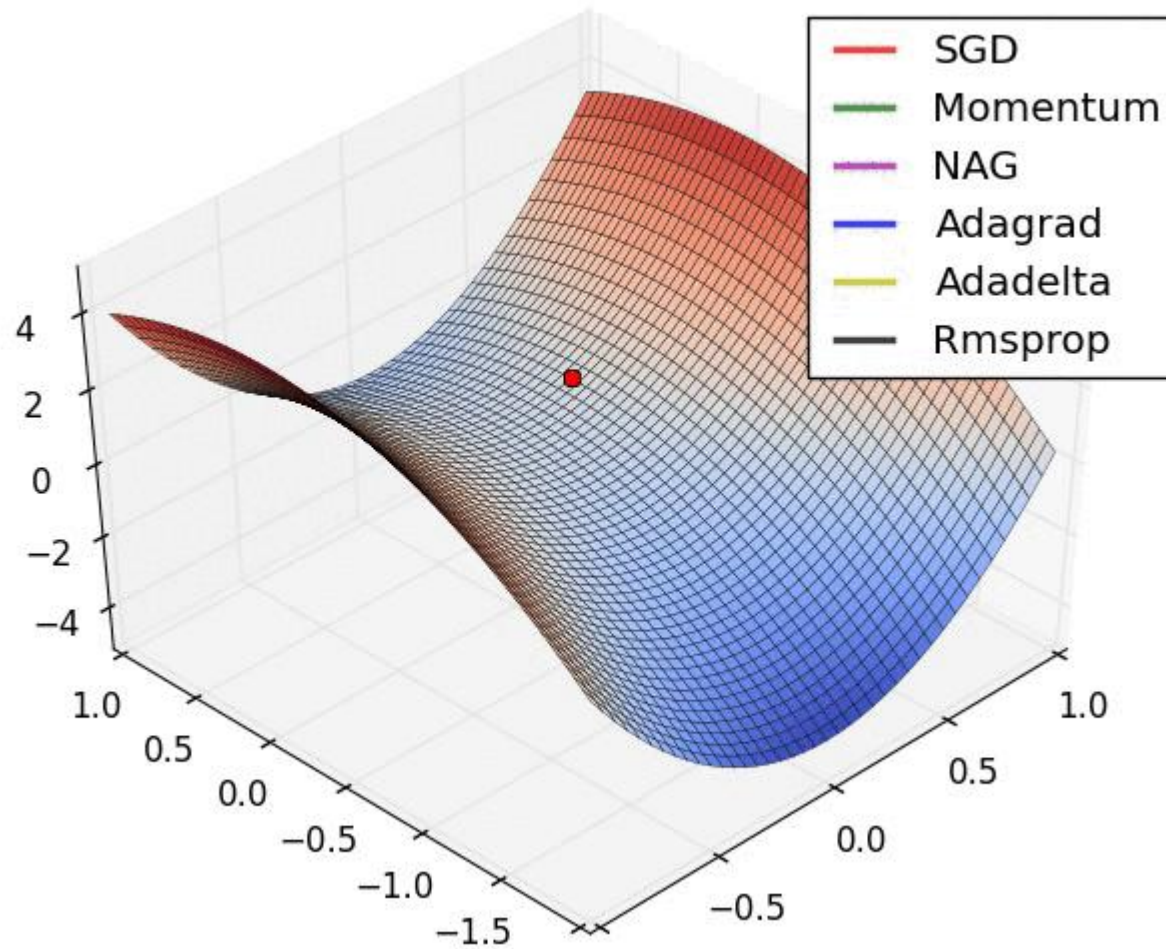
- Idea: Normalize step size by average gradient magnitude in each direction
- Examples: RMSProp, Adagrad
- **Adam** = Momentum + RMSProp
 - Commonly used today

Comparison of SGD Variants



- SGD: Converges slowly
- Momentum: Moves much faster, although can overshoot
- Adagrad, RMSProp: Converge quickly here

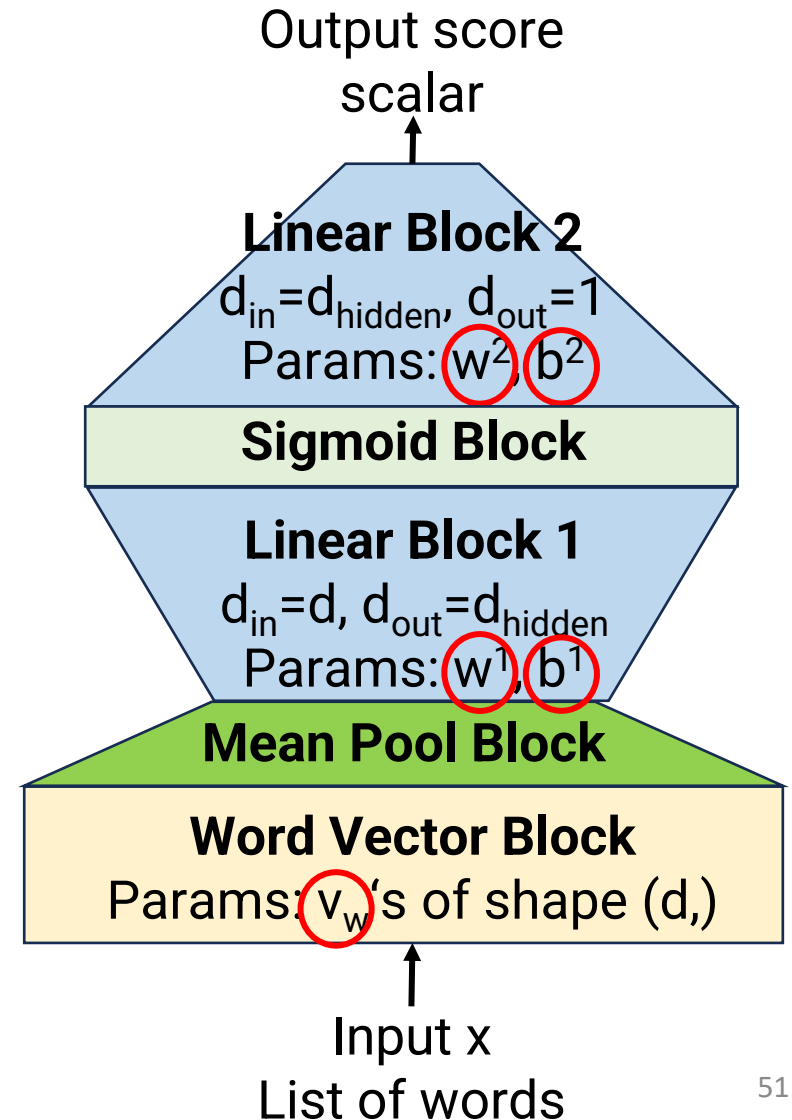
Comparison of SGD Variants



- One problem for non-convex optimization:
Saddle points
 - Function is locally flat but is neither local minimum nor local maximum
 - Need to “escape” these to find a minimum
- SGD gets stuck, other methods can escape

Computing Gradients

- How to compute gradient of loss with respect to all the parameters?
- We could do it by hand (as we did for Logistic Regression)
- But there is also an algorithmic way: **Backpropagation**
- Key idea: Use chain rule to differentiate through each “block”
 - We can leverage the modular structure of neural networks here!
 - This is what pytorch does for us; more on this in a couple classes



Conclusion

- Neural Networks are powerful machine learning models!
- Combined with word vectors, we can use them to build NLP models
- With CNN's, we can learn to process fixed-sized windows of words
- Next: Beyond fixed size